

Objetivos da aula:

- Calcular determinantes e interpretar seu significado geométrico e computacional.
- Identificar autovalores e autovetores e compreender suas interpretações.
- Relacionar autovalores ao comportamento de uma transformação linear.
- Verificar e aplicar a diagonalização de matrizes.
- Utilizar NumPy para explorar determinantes, autovalores e autovetores.
- Analisar transformações lineares por meio de suas direções principais.

Índice:

- I. Motivação computacional
- II. Determinantes: Definições Formais e Interpretação Geométrica
- III. Autovalores e Autovetores (Eigenvalues & Eigenvectors)
- IV. Diagonalização de Matrizes

i. Motivação computacional

Na anterior, aprendemos que as matrizes não são apenas tabelas de dados estáticas; elas são **máquinas** (operadores) que transformam o espaço vetorial. Elas esticam, giram, cizalham e projetam vetores.

No entanto, no mundo real da Ciência de Dados e Engenharia de ML, raramente lidamos com matrizes 2×2 ou 3×3 que podemos inspecionar visualmente. Trabalhamos frequentemente com matrizes gigantescas (milhões de linhas e colunas).

Surge então uma necessidade crítica:

Como resumir o comportamento de uma matriz complexa em apenas alguns números?

Assim como um médico não analisa cada célula de um paciente, mas pede exames de sangue para obter "indicadores" gerais de saúde, na Álgebra Linear usamos o **Determinante** e os **Autovalores/Autovetores** para fazer uma "análise interna" da matriz. Eles revelam o **DNA da transformação linear**.

Vamos entender por que estes conceitos são vitais para a computação moderna.

1. O "Volume" da Informação (Determinante)

Quando uma rede neural processa dados, ela aplica sucessivas transformações lineares. Uma pergunta fundamental é: Essa transformação preserva ou destrói a informação?

- O Fator de Escala: O determinante diz-nos exatamente o quanto a matriz "escala" o espaço. Se $\det(A) = 2$, a transformação dobra o volume dos dados. Se $\det(A) = 0.5$, ela comprime os dados à metade.
- O Colapso da Dimensão (Singularidade): O caso mais crítico em computação é quando $\det(A) = 0$.
 - Geometricamente, isso significa que a matriz "achatou" o espaço (transformou um cubo 3D num plano plano, ou um plano numa linha).

- Em Data Science, isso indica redundância: as colunas da matriz são linearmente dependentes (LD).
- Em Sistemas Lineares, isso indica que a matriz não tem inversa, o que pode fazer algoritmos numéricos falharem (erro de *Singular Matrix*).

Saber calcular o determinante ajuda-nos a diagnosticar se o nosso modelo matemático é estável ou colapsa a informação.

2. Os Eixos Principais (Autovalores e Autovetores)

Se aplicarmos uma rotação ou distorção aleatória a uma imagem, quase todos os vetores mudam de direção. Mas existem alguns vetores especiais que **não saem dos eixos**, apenas esticam ou encolhem.

- **Autovetores (Eigenvectors):** São as direções "preferenciais" da matriz. Eles indicam os eixos onde a transformação age de forma pura.
- **Autovalores (Eigenvalues):** São os números que dizem a intensidade dessa ação (o quanto estica ou encolhe nessa direção).

3. Simplificando o Complexo (Diagonalização)

Por fim, o "Santo Graal" de muitas operações matriciais é a Diagonalização. Matrizes diagonais (zeros fora da diagonal principal) são computacionalmente perfeitas: são fáceis de inverter, fáceis de multiplicar e fáceis de elevar a potências.

Diagonalizar uma matriz significa encontrar uma mudança de base (usando autovetores) que transforma uma matriz complicada e cheia de números numa matriz diagonal limpa. Isso permite-nos, por exemplo, prever estados futuros de sistemas dinâmicos (como a evolução de uma pandemia ou o tempo) de forma muito eficiente.

ii. Determinantes: Definições Formais e Interpretação Geométrica

O determinante é um dos conceitos centrais da Álgebra Linear. Embora muitas vezes seja introduzido por meio de regras de cálculo mecânicas, o seu significado profundo está ligado à geometria da transformação linear e à análise estrutural de matrizes.

1. Definições Formais do Determinante

a. Determinante de Matrizes 2x2

Para uma matriz

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

o determinante é definido por:

$$\det(A) = ad - bc$$

Esta é a forma mais simples e será o ponto de partida para compreensões posteriores.

Em Python:

```
import numpy as np

A = np.array([[3, 2],
              [1, 4]])

det_A_manual = A[0,0]*A[1,1] - A[0,1]*A[1,0] # Cálculo manual
det_A_numpy = np.linalg.det(A) # usando NumPy

print("Determinante (manual):", det_A_manual)
print("Determinante (NumPy): ", det_A_numpy)
```

Saída:

```
Determinante (manual): 10
Determinante (NumPy): 10.0000000000000002
```

b. Determinante de Matrizes 3x3

Para uma matriz

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} \\ a_{21} & a_{22} & a_{23} \\ a_{31} & a_{32} & a_{33} \end{bmatrix}$$

uma definição comum utiliza a regra de Sarrus:

$$\det(A) = a_{11}a_{22}a_{33} + a_{12}a_{23}a_{31} + a_{13}a_{21}a_{32} - a_{13}a_{22}a_{31} - a_{11}a_{23}a_{32} - a_{12}a_{21}a_{33}$$

Em Python:

```
A = np.array([[1, 2, 3],
              [0, 1, 4],
              [5, 6, 0]])

# Cálculo manual (Sarrus)
det_manual = (
    A[0,0]*A[1,1]*A[2,2] +
    A[0,1]*A[1,2]*A[2,0] +
    A[0,2]*A[1,0]*A[2,1] -
    A[0,2]*A[1,1]*A[2,0] -
    A[0,0]*A[1,2]*A[2,1] -
    A[0,1]*A[1,0]*A[2,2]
)

det_numpy = np.linalg.det(A) # usando NumPy
print("Determinante (manual via Sarrus):", det_manual)
print("Determinante (NumPy): ", det_numpy)
```

Saída:

```
Determinante (manual via Sarrus): 1
Determinante (NumPy): 0.99999999999999964
```

c. Expansão por Cofatores (Definição Geral)

Para uma matriz quadrada de ordem n , o determinante é definido recursivamente:

$$\det(A) = \sum_{j=1}^n (-1)^{1+j} a_{1j} \det(A_{1j})$$

onde:

- A_{1j} é a matriz obtida ao remover a linha 1 e a coluna j ,
- $(-1)^{1+j}$ é o sinal associado ao cofator.

A regra de Sarrus:

- Só funciona para matrizes 3×3 ;
- É um atalho algébrico, uma fórmula direta obtida da expansão por cofatores aplicada especificamente ao caso 3×3 ;
- Consiste em repetir as duas primeiras colunas da matriz e somar os produtos das diagonais principais, subtraindo os das diagonais secundárias.

A expansão por cofatores:

- Funciona para qualquer matriz $n \times n$, de qualquer tamanho;
- É a **definição formal e recursiva** do determinante;
- É usada para *definir* o determinante, não apenas para calculá-lo;
- Possibilita a prova de todas as propriedades do determinante.

Em termos computacionais (NumPy):

- NumPy não usa Sarrus ou cofatores generalizados;
- NumPy usa decomposições numéricas avançadas para melhor precisão e desempenho.

Assim, o valor "0.9999999999999964" é apenas um erro de arredondamento numérico.

2. Propriedades Fundamentais do Determinante

Estas propriedades conferem ao determinante o seu poder como ferramenta algébrica e computacional.

1. Efeito das operações elementares:

- Trocar duas linhas multiplica o determinante por -1 .

Resultado esperado: $\det(B) = -\det(A)$

```
A = np.array([[2, 1],
              [3, 4]])
B = A[[1,0]] # Troca de linhas
print(f"B:{B}")
print("det(A) =", np.linalg.det(A))
print("det(B) =", np.linalg.det(B))
```

Saída:

```
B: [[3 4]
     [2 1]]
det(A) = 5.000000000000001
det(B) = -5.000000000000001
```

- Multiplicar uma linha por k multiplica o determinante por k .

Cuidado: Multiplicar a matriz inteira (todas as linhas) por k escala o determinante por k^n . Exemplo:

Em 2D, dobrar o tamanho da matriz ($2A$) quadruplica a área ($2^2 = 4$).

```
A = np.array([[2, 1],
              [3, 4]])
B = A.copy()
B[1] = 5 * B[1]    # Multiplica a segunda linha por 5
print(f"B:{B}")
print("det(A)      =", np.linalg.det(A))
print("det(5 · L2(A)) =", np.linalg.det(B))
print("5 × det(A)   =", 5 * np.linalg.det(A))
```

Saída:

```
B: [[ 2  1]
     [15 20]]
det(A)      = 5.0000000000000001
det(5 · L2(A)) = 24.999999999999996
5 × det(A)   = 25.000000000000004
```

- Somar um múltiplo de uma linha a outra **não altera** o determinante.

```
import numpy as np
A = np.array([[2, 1],
              [3, 4]])
B = A.copy()
B[1] = B[1] + 10 * B[0]    # L2 ← L2 + 10 · L1
print(f"B:{B}")
print("det(A) =", np.linalg.det(A))
print("det(B) =", np.linalg.det(B))
```

Saída:

```
B: [[ 2  1]
     [23 14]]
det(A) = 5.0000000000000001
det(B) = 5.0000000000000002
```

2. Determinante de matrizes triangulares:

$\det(A)$ = produto dos elementos da diagonal principal

```
import numpy as np
A = np.array([[2, 3, 1],
              [0, 5, 4],
              [0, 0, 7]])
det_numpy = np.linalg.det(A)
det_manual = A[0,0] * A[1,1] * A[2,2] # 2 * 5 * 7
print("Determinante (manual):", det_manual)
print("Determinante (NumPy): ", det_numpy)
```

Saída:

```
Determinante (manual): 70
Determinante (NumPy): 69.999999999999996
```

3. Compatibilidade com o produto matricial:

$$\det(AB) = \det(A) \det(B)$$

Esta é uma das propriedades mais importantes do determinante, pois mostra que o determinante é um **invariante multiplicativo** associado às transformações lineares representadas por matrizes.

```
import numpy as np
A = np.array([[2, 1],
              [3, 4]])
B = np.array([[1, 2],
              [0, 5]])
AB = A @ B # Produto matricial
det_A = np.linalg.det(A)
det_B = np.linalg.det(B)
det_AB = np.linalg.det(AB)
print("det(A) =", det_A)
print("det(B) =", det_B)
print("det(AB) =", det_AB)
print("det(A)*det(B) =", det_A * det_B)
```

Saída:

```
det(A) = 5.0000000000000001
det(B) = 4.999999999999999
det(AB) = 25.000000000000007
det(A)*det(B) = 25.0
```

4. Determinante e inversa:

$$A \text{ é inversível} \Leftrightarrow \det(A) \neq 0, \quad \det(A^{-1}) = \frac{1}{\det(A)}$$

```
import numpy as np
A = np.array([[4, 2],
              [3, 1]])

det_A = np.linalg.det(A)

print("det(A) =", det_A)

# Verifica se A é inversível
if abs(det_A) > 1e-12:
    A_inv = np.linalg.inv(A)
    det_A_inv = np.linalg.det(A_inv)

    print("A é inversível.")
    print("det(A^{-1}) =", det_A_inv)
    print("1/det(A) =", 1/det_A)
else:
    print("A não é inversível (determinante zero).")
```

Saída:

```
det(A) = -2.0
A é inversível.
det(A^{-1}) = -0.5000000000000001
```

$$1/\det(A) = -0.5$$

Interpretação

- O determinante não nulo garante que a transformação linear *não colapsa o espaço*.
- A matriz inversa existe e seu determinante é o inverso do determinante original.

3. O Determinante como Fator de Escala (Interpretação Geométrica)

A interpretação geométrica do determinante liga a definição algébrica ao modo como matrizes transformam o espaço.

Fator de Zoom

Uma matriz A aplicada a uma região do espaço (como um quadrado ou cubo unitário) produz uma região transformada. O determinante indica como o **tamanho** dessa região muda:

- $\det(A) = 2$: a área/volume dobra;
- $|\det(A)| = 0.5$: reduz-se à metade;
- $\det(A) = 1$: preserva a área/volume (ex.: rotações);
- $\det(A) = 0$: colapso dimensional.

Exemplos:

- Matriz que dobra a área: Qualquer figura no plano tem sua área dobrada.

```
A = np.array([[2, 0],
              [0, 1]])
print("det(A) =", np.linalg.det(A))
```

Saída:

```
det(A) = 2.0
```

- Rotação preserva área

```
theta = np.radians(30)
R = np.array([[np.cos(theta), -np.sin(theta)],
              [np.sin(theta),  np.cos(theta)]])
print(f"R: {R}")
print("det(R) =", np.linalg.det(R))
```

Saída:

```
R: [[ 0.8660254 -0.5
      [ 0.5      0.8660254]]]
det(R) = 1.0
```

- Matriz singular (colapso dimensional): A matriz achata o plano; não há inversa.

```
C = np.array([[1, 2],
              [2, 4]])
print("det(C) =", np.linalg.det(C))
```

Saída:

```
det(C) = 0.0
```

4. O Sinal do Determinante (Orientação)

- $\det(A) > 0$: preserva a orientação.
- $\det(A) < 0$: inverte a orientação (espelhamento).

O **módulo** do determinante dá o fator de escala; o **sinal** informa se o espaço foi “virado”.

5. Implementação Computacional

Este exemplo ajuda a ilustrar como a singularidade “esmaga” o plano.

A imagem mostra que os dois vetores colapsam na mesma direção, confirmando a dependência linear.

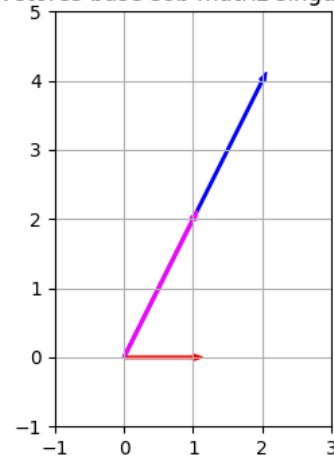
```
import matplotlib.pyplot as plt

# Vetores base do plano
v1 = np.array([1, 0])
v2 = np.array([0, 1])
C = np.array([[1, 2],
              [2, 4]])

v1_t = C @ v1
v2_t = C @ v2

plt.figure(figsize=(3, 4))
plt.arrow(0,0,*v1, width=0.03, color="red")
plt.arrow(0,0,*v2_t, width=0.03, color="blue")
plt.arrow(0,0,*v1_t, width=0.03, color="magenta")
plt.xlim(-1,3)
plt.ylim(-1,5)
plt.gca().set_aspect('equal')
plt.title("Vetores base sob matriz singular C")
plt.grid(True)
plt.show()
```

Vetores base sob matriz singular C



iii. Autovalores e Autovetores (Eigenvalues & Eigenvectors)

Os autovalores e autovetores constituem uma das estruturas mais fundamentais da Álgebra Linear, pois revelam a “anatomia interna” das transformações lineares representadas por matrizes. Se o determinante fornece um único número que resume o fator de escala global da transformação, os autovalores e autovetores descrevem **como essa transformação se comporta em direções específicas do espaço**.

Do ponto de vista computacional, esses conceitos são intrínsecos a áreas como compressão de imagens, redução de dimensionalidade, estabilidade de sistemas dinâmicos, aprendizado de máquina, análise espectral, recomendação e otimização. A compreensão de autovalores não é apenas teórica — ela é essencial para interpretar e controlar algoritmos modernos.

1. Intuição Geométrica

Considere uma transformação linear $T: R^n \rightarrow R^n$ representada por uma matriz A .

Em geral, ao aplicarmos a matriz a um vetor v , sua **direção é alterada**:

$$A v \neq \lambda v$$

No entanto, existem vetores muito especiais — os **autovetores** — que não mudam de direção após a aplicação da transformação. Eles podem ser esticados, comprimidos ou invertidos, mas **continuam apontando para a mesma reta**.

Definição Fundamental

Um vetor não-nulo v é autovetor de A se existir um número real (ou complexo) λ tal que:

$$A v = \lambda v$$

O número λ é o **autovalor** associado ao autovetor v .

Este simples enunciado contém profunda informação estrutural:

- v indica uma **direção invariável** pela transformação.
- λ indica o **fator de escala** dessa direção.

Transformações lineares podem deformar o espaço em diversas direções, mas os autovetores indicam suas “direções puras”.

2. A Equação Característica

Partindo de

$$A v = \lambda v$$

podemos reorganizar:

$$(A - \lambda I)v = 0$$

Para que existam soluções não triviais $v \neq 0$, devemos ter:

$$\det(A - \lambda I) = 0$$

Esta equação é o **polinômio característico** de A , cujas raízes são os autovalores.

Grau do Polinômio

Se $A \in R^{n \times n}$, então o polinômio característico é de grau n , isto é:

$$p(\lambda) = \det(A - \lambda I)$$

Consequentemente, toda matriz quadrada possui até n autovalores, reais ou complexos.

3. Autovalores em Matrizes 2x2

Para uma matriz

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

o polinômio característico é:

$$\det(A - \lambda I) = \det \begin{bmatrix} a - \lambda & b \\ c & d - \lambda \end{bmatrix} = (a - \lambda)(d - \lambda) - bc$$

Expansão:

$$\lambda^2 - (a + d)\lambda + (ad - bc) = 0$$

Observe que:

- $a + d = \text{tr}(A)$ (traço da matriz),
- $ad - bc = \det(A)$.

Assim, para matrizes 2x2:

$$\lambda^2 - \text{tr}(A)\lambda + \det(A) = 0$$

Esta relação permite verificar propriedades:

- A soma dos autovalores é igual ao traço.
- O produto dos autovalores é igual ao determinante.

4. Implementação Computacional

O NumPy computa autovalores e autovetores por meio da função `np.linalg.eig`.

```
import numpy as np

A = np.array([[4, 2],
              [1, 3]])

vals, vecs = np.linalg.eig(A)

print("Autovalores:")
print(vals)
print("\nAutovetores (colunas):")
print(vecs)

# Verificação do conceito Av = λv
v = vecs[:, 0]
lambda_ = vals[0]

print("\nAv:")
print(A @ v)
print("\nλv:")
print(lambda_ * v)
```

Saída:

Autovalores:
[5. 2.]

Autovetores (colunas):
[[0.89442719 -0.70710678]
 [0.4472136 0.70710678]]

Av:
[4.47213595 2.23606798]

λv:
[4.47213595 2.23606798]

Interpretação:

- Cada coluna de `vecs` é um autovetor.
- O cálculo confirma numericamente que $Av = \lambda v$.

Nota: O NumPy retorna os autovetores normalizados (comprimento 1). Se você calcular na mão, poderá encontrar vetores proporcionais (ex: [1, 2]), que também estão corretos.

5. Interpretação Geométrica no Plano ($\mathbb{R}^2$)

Para matrizes 2×2 , os autovetores podem ser visualizados diretamente:

- Se $\lambda > 1$, aquela direção é esticada.
- Se $0 < \lambda < 1$, a direção é comprimida.
- Se $\lambda < 0$, há inversão de orientação (reflexão).
- Se não houver autovalores reais, a matriz representa rotação ou combinação que não preserva direção real.

Exemplo ilustrativo:

```
import numpy as np
import matplotlib.pyplot as plt

def plot_eigen_verification():
    # 1. Definição da Matriz A
    # Usamos uma matriz simétrica para garantir autovetores ortogonais (mais fácil de ver)
```

```
A = np.array([[3, 1],
              [1, 3]])

# 2. Cálculo dos Autovalores e Autovetores
vals, vecs = np.linalg.eig(A)

# Dica de ML: Ordenar do maior para o menor autovalor (importante para PCA)
idx = vals.argsort()[::-1]
vals = vals[idx]
vecs = vecs[:, idx]

# Configuração do gráfico
fig, ax = plt.subplots(figsize=(5, 5))
# Cores para diferenciar os pares
colors = ['#FF0000', '#0000FF'] # Vermelho e Azul
origin = [0, 0]
print(f"Matriz A:\n{A}\n")

# 3. Loop para plotar cada autovetor e sua transformação
for i in range(len(vals)):
    lambda_val = vals[i]
    v = vecs[:, i] # Autovetor original (normalizado pelo NumPy)
    Av = A @ v     # Vetor transformado (esticado por lambda)

    color = colors[i]
    print(f"--- Par {i+1} ---")
    print(f"Autovalor (lambda): {lambda_val:.2f}")
    print(f"Autovetor (v): {v}")
    print(f"Transformado (Av): {Av}")
    print(f"Verificação: Av == lambda * v? {np.allclose(Av, lambda_val * v)}\n")

    # A) Plotar o Vetor transformado (Av) -> Seta Grossa e Transparente
    # Isso representa o resultado da transformação (esticamento)
    ax.quiver(*origin, Av[0], Av[1], color=color, alpha=0.3,
              angles='xy', scale_units='xy', scale=1,
              width=0.025, label=f'Transformado Av ( $\lambda$ = {lambda_val:.1f})' if i==0
    else "")

    # B) Plotar o Autovetor original (v) -> Seta Fina e Sólida
    # Isso representa a direção original
    ax.quiver(*origin, v[0], v[1], color=color, alpha=1.0,
              angles='xy', scale_units='xy', scale=1,
              width=0.008, label=f'Original v' if i==0 else "")

    # Adicionar anotação de texto para identificar o autovalor
    offset = 0.2
    ax.text(Av[0] + offset, Av[1], f" $\lambda$ = {lambda_val:.0f}", color=color,
            fontsize=12, fontweight='bold')
```

```
# 4. Adicionar um vetor "Comum" (NÃO autovetor) para contraste
# Para mostrar que um vetor qualquer rodaria e sairia do alinhamento
v_random = np.array([1, 0])
Av_random = A @ v_random

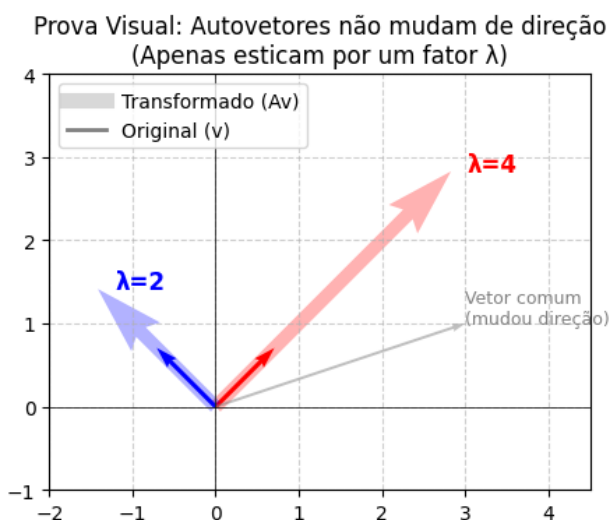
# Plota vetor não-eigen transformado (tracejado) para mostrar que ele gira
ax.quiver(*origin, Av_random[0], Av_random[1], color='gray', alpha=0.5,
          angles='xy', scale_units='xy', scale=1, width=0.005, linestyle='--')
ax.text(Av_random[0], Av_random[1], "Vetor comum\n(mudou direção)", color='gray',
        fontsize=9)

# Ajustes finais do gráfico
limit = 4.5
ax.set_xlim(-limit, limit)
ax.set_ylim(-limit, limit)
ax.axhline(0, color='black', linewidth=0.5)
ax.axvline(0, color='black', linewidth=0.5)
ax.grid(True, linestyle='--', alpha=0.6)
ax.set_aspect('equal')
ax.set_title(f"Prova Visual: Autovetores não mudam de direção\n(Apenas esticam por um fator  $\lambda$ )")

# Criar legenda personalizada para explicar a sobreposição
from matplotlib.lines import Line2D
custom_lines = [Line2D([0], [0], color='gray', lw=8, alpha=0.3,
label='Transformado (Av)'),
                Line2D([0], [0], color='gray', lw=2, label='Original (v)')]
ax.legend(handles=custom_lines, loc='upper left')

plt.show()

plot_eigen_verification()
```



Interpretação:

- A Sobreposição: Note que a seta fina (escura) está perfeitamente centralizada dentro da seta grossa (clara). Não há rotação.
- O Comprimento: A seta grossa é exatamente λ vezes maior que a fina.
 - Para $\lambda = 4$, a seta grossa é 4x maior.
 - Para $\lambda = 2$, é 2x maior.
- O Contraste: Adicionei um "Vetor comum" cinza no gráfico. Note que ele não tem essa

Saída:

Matriz A:

```
[[3 1]
 [1 3]]
```

--- Par 1 ---

```
Autovalor (lambda): 4.00
Autovetor (v): [0.70710678 0.70710678]
Transformado (Av): [2.82842712
2.82842712]
Verificação: Av == lambda * v? True
```

--- Par 2 ---

```
Autovalor (lambda): 2.00
Autovetor (v): [-0.70710678 0.70710678]
Transformado (Av): [-1.41421356
1.41421356]
Verificação: Av == lambda * v? True
```

propriedade: a transformação mudou a direção dele (ele não está alinhado com o eixo x original).
Isso valoriza a "magia" dos autovetores.

6. Propriedades Essenciais dos Autovalores

1. O traço da matriz é a soma dos autovalores:

$$\text{tr}(A) = \sum \lambda_i$$

2. O determinante da matriz é o produto dos autovalores:

$$\det(A) = \prod \lambda_i$$

3. Matrizes triangulares têm autovalores iguais aos elementos da diagonal.
4. Matrizes simétricas reais possuem:

- autovalores reais,
- autovetores ortogonais,
- decomposição espectral particularmente estável numericamente.

iv. Diagonalização de Matrizes

A diagonalização é um dos resultados mais poderosos da Álgebra Linear, pois permite transformar uma matriz complexa em uma matriz extremamente simples — uma matriz diagonal — por meio de uma mudança adequada de base. Essa simplificação torna operações matriciais muito mais eficientes e revela a estrutura interna da transformação linear.

Do ponto de vista computacional, diagonalizar uma matriz permite:

- acelerar cálculos de potências de matrizes,
- analisar estabilidade de sistemas dinâmicos,
- decompor transformações em direções coordenadas,
- compreender algoritmos baseados em autovalores (como PCA e análise espectral).

1. Motivação: Por que diagonalizar?

Operações com matrizes gerais podem ser custosas. Já uma matriz diagonal:

$$D = \begin{bmatrix} \lambda_1 & 0 & \cdots & 0 \\ 0 & \lambda_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \lambda_n \end{bmatrix}$$

é extremamente simples de manipular:

- Elevar D à potência k é imediato:

$$D^k = \begin{bmatrix} \lambda_1^k & \cdots & 0 \\ \vdots & \ddots & \vdots \\ 0 & \cdots & \lambda_n^k \end{bmatrix}$$

- A inversa de D existe sempre que $\lambda_i \neq 0$:

$$D^{-1} = \text{diag}\left(\frac{1}{\lambda_1}, \dots, \frac{1}{\lambda_n}\right)$$

- Multiplicações e transformações se tornam extremamente rápidas e estáveis.

Assim, diagonalizar uma matriz significa reduzir uma transformação linear complexa a várias transformações simples e independentes.

2. Definição Formal

Uma matriz $A \in R^{n \times n}$ é diagonalizável se existir uma matriz inversível P e uma matriz diagonal D tais que:

Onde:

$$A = PDP^{-1}$$

- As colunas de P são autovetores de A ;
- Os elementos da diagonal de D são os autovalores correspondentes.

Em outras palavras:

Diagonalizar é **mudar a base** para aquela em que A age da maneira mais simples possível: escalamentos independentes.

3. Condição para Diagonalização

Uma matriz é diagonalizável se, e somente se, possui um conjunto de **n autovetores linearmente independentes**.

Isso equivale à condição:

$$\text{Multiplicidade geométrica} = \text{Multiplicidade algébrica} \quad \text{para cada autovalor.}$$

- Multiplicidade algébrica: quantas vezes o autovalor aparece como raiz do polinômio característico.
- Multiplicidade geométrica: dimensão do espaço nulo de $A - \lambda I$.

Consequências:

- Matrizes com autovalores distintos **sempre** são diagonalizáveis.
- Matrizes simétricas reais **sempre** são diagonalizáveis e possuem autovetores ortogonais.
- Matrizes com poucos autovetores independentes **não** são diagonalizáveis (caso típico: matrizes de Jordan).

4. Processo de Diagonalização

Para diagonalizar uma matriz A :

1. Encontre os autovalores de A .
2. Encontre os autovetores correspondentes a cada autovalor.
3. Construa a matriz

$$P = [v_1 \ v_2 \ \dots \ v_n]$$

onde cada v_i é um autovetor.

4. Construa a matriz

$$D = \text{diag}(\lambda_1, \dots, \lambda_n)$$

5. Verifique:

$$A = PDP^{-1}$$

5. Exemplo Computacional (NumPy)

Considere a matriz:

```
import numpy as np

A = np.array([[4, 1],
              [1, 4]])
```

Cálculo dos autovalores e autovetores

```
vals, vecs = np.linalg.eig(A)

print("Autovalores:", vals)
print("Autovetores (colunas de P):")
print(vecs)
```

Saída:

```
Autovalores: [5. 3.]
Autovetores (colunas de P):
[[ 0.70710678 -0.70710678]
 [ 0.70710678  0.70710678]]
```

Construindo P , D e verificando $A = PDP^{-1}$

```
P = vecs
D = np.diag(vals)

A_rec = P @ D @ np.linalg.inv(P)

print("A reconstruída:")
print(A_rec)
```

Saída:

```
A reconstruída:
[[4. 1.]
 [1. 4.]]
```

O resultado exibirá uma matriz igual (ou numericamente muito próxima) à matriz original A , confirmando a diagonalização.

6. Exemplo: Matriz Não Diagonalizável

Considere:

$$A = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

Essa matriz possui apenas **um** autovalor ($\lambda = 1$) e **um único autovetor** independente. Logo, ela **não é diagonalizável**.

Em NumPy:

```
A = np.array([[1, 1],
              [0, 1]])

vals, vecs = np.linalg.eig(A)

print("Autovalores:", vals)
print("Autovetores:")
print(vecs)
```

Saída:

```
Autovalores: [1. 1.]
Autovetores:
[[ 1.00000000e+00 -1.00000000e+00]
 [ 0.00000000e+00  2.22044605e-16]]
```

O algoritmo mostrará apenas **um autovetor linearmente independente**, impossibilitando a construção de uma matriz P invertível.

Exercícios

Determinantes e Interpretação Geométrica – Exercícios 1 a 4

Exercício 1 – Determinante 2x2 e fator de escala

Considere a matriz

$$A = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix}$$

- Calcule $\det(A)$.
- Interprete geometricamente o resultado: o que acontece com a área de um quadrado unitário após a transformação por A ?

Resposta:

- Para uma matriz 2x2,

$$\det(A) = ad - bc = 3 \cdot 2 - 0 \cdot 0 = 6$$

- Geometricamente, o determinante é o fator de escala da área.

Um quadrado unitário (área = 1) é transformado em um paralelogramo com área:

$$\{\text{área nova}\} = |\det(A)| \cdot \{\text{área original}\} = 6 \cdot 1 = 6$$

Interpretação: a transformação multiplica a área por 6.

Exercício 2 – Determinante 3x3 (Regra de Sarrus)

Calcule o determinante da matriz

$$B = \begin{bmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{bmatrix}$$

usando a regra de Sarrus.

Resposta:

Pela regra de Sarrus, para

$$B = \begin{bmatrix} b_{11} & b_{12} & b_{13} \\ b_{21} & b_{22} & b_{23} \\ b_{31} & b_{32} & b_{33} \end{bmatrix}$$

temos:

$$\det(B) = b_{11}b_{22}b_{33} + b_{12}b_{23}b_{31} + b_{13}b_{21}b_{32} - b_{13}b_{22}b_{31} - b_{11}b_{23}b_{32} - b_{12}b_{21}b_{33}$$

Identificando:

- $b_{11} = 1, b_{12} = 2, b_{13} = 0$
- $b_{21} = 0, b_{22} = 1, b_{23} = 3$
- $b_{31} = 2, b_{32} = 0, b_{33} = 1$

Agora:

1. Termos positivos:

- $b_{11}b_{22}b_{33} = 1 \cdot 1 \cdot 1 = 1$
- $b_{12}b_{23}b_{31} = 2 \cdot 3 \cdot 2 = 12$
- $b_{13}b_{21}b_{32} = 0 \cdot 0 \cdot 0 = 0$

Soma dos positivos: $1 + 12 + 0 = 13$.

2. Termos negativos:

- $b_{13}b_{22}b_{31} = 0 \cdot 1 \cdot 2 = 0$
- $b_{11}b_{23}b_{32} = 1 \cdot 3 \cdot 0 = 0$
- $b_{12}b_{21}b_{33} = 2 \cdot 0 \cdot 1 = 0$

Soma dos negativos: $0 + 0 + 0 = 0$.

Logo, $\det(B) = 13 - 0 = 13$.

Exercício 3 – Operações elementares e determinante

Considere

$$C = \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix}$$

Construa a matriz C' obtida de C pela operação:

$$L_2 \leftarrow L_2 + 5L_1, \text{ onde } L_1 = (2,1) \text{ e } L_2 = (3,4).$$

- a) Calcule $\det(C)$.
- b) Calcule $\det(C')$.
- c) O que isso mostra sobre o efeito de somar um múltiplo de uma linha a outra no determinante?

Resposta:

a) Determinante de C :

$$\det(C) = 2 \cdot 4 - 1 \cdot 3 = 8 - 3 = 5$$

b) Aplicando a operação:

$$L'_2 = L_2 + 5L_1 = (3,4) + 5(2,1) = (3 + 10, 4 + 5) = (13,9)$$

Assim,

$$C' = \begin{bmatrix} 2 & 1 \\ 13 & 9 \end{bmatrix}$$

Determinante de C' :

$$\det(C') = 2 \cdot 9 - 1 \cdot 13 = 18 - 13 = 5$$

c) Conclusão: $\det(C') = \det(C)$

Portanto, somar um múltiplo de uma linha a outra linha não altera o determinante.

Exercício 4 – Determinante zero e dependência linear

Considere

$$D = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}$$

- a) Calcule $\det(D)$.
- b) Verifique se as colunas são linearmente dependentes.
- c) Relacione o resultado com o fato de a matriz ser singular.

Resposta:

a) $\det(D) = 1 \cdot 4 - 2 \cdot 2 = 4 - 4 = 0$

b) Colunas: $c_1 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$ e $c_2 = \begin{bmatrix} 2 \\ 4 \end{bmatrix} = 2 \cdot c_1$

Logo, as colunas são linearmente dependentes, pois c_2 é múltiplo de c_1 .

- c) Como o determinante é zero, a matriz é singular (não tem inversa) e ocorre colapso dimensional: o plano é “achatado” em uma única reta. Isso está coerente com a dependência linear das colunas.

Autovalores e Autovetores – Exercícios 5 a 7

Exercício 5 – Autovalores e autovetores de uma matriz 2x2

Considere

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$$

- a) Encontre os autovalores de A .
- b) Para cada autovalor, encontre um autovetor associado.

Resposta:

- a) Polinômio característico:

$$\det(A - \lambda I) = \det \begin{bmatrix} 4 - \lambda & 1 \\ 1 & 4 - \lambda \end{bmatrix} = (4 - \lambda)^2$$

Expansão:

$$(4 - \lambda)^2 - 1 = (16 - 8\lambda + \lambda^2) - 1 = \lambda^2 - 8\lambda + 15$$

Resolver:

$$\lambda^2 - 8\lambda + 15 = 0$$

Fatorando:

$$(\lambda - 3)(\lambda - 5) = 0 \Rightarrow \lambda_1 = 3, \lambda_2 = 5$$

b) Autovetores:

- Para $\lambda_1 = 3$:

$$(A - 3I)v = 0 \Rightarrow \begin{bmatrix} 4-3 & 1 \\ 1 & 4-3 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Sistema:

$$x + y = 0 \Rightarrow y = -x$$

Um autovetor possível:

$$v_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$$

- Para $\lambda_2 = 5$:

Sistema:

$$-x + y = 0 \Rightarrow y = x$$

Um autovetor possível:

$$v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

Exercício 6 – Traço, determinante e autovalores

Use a matriz do exercício anterior,

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$$

e os autovalores $\lambda_1 = 3, \lambda_2 = 5$.

- Calcule o traço de A e compare com $\lambda_1 + \lambda_2$.
- Calcule o determinante de A e compare com $\lambda_1 \cdot \lambda_2$.

Resposta:

- Traço:

$$\text{tr}(A) = 4 + 4 = 8$$

Soma dos autovalores:

$$\lambda_1 + \lambda_2 = 3 + 5 = 8$$

Logo,

$$\text{tr}(A) = \lambda_1 + \lambda_2$$

- Determinante:

$$\det(A) = 4 \cdot 4 - 1 \cdot 1 = 16 - 1 = 15$$

Produto dos autovalores:

$$\lambda_1 \lambda_2 = 3 \cdot 5 = 15$$

Conclusão:

$$\det(A) = \lambda_1 \lambda_2$$

Exercício 7 – Interpretação dinâmica simples

Considere o sistema discreto

$$x_{k+1} = Bx_k$$

com

$$B = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}, \quad x_0 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- Quais são os autovalores de B ?
- Qual o comportamento de x_k para $k \rightarrow \infty$ em relação a cada componente?

Resposta:

- B é diagonal, logo os autovalores são os elementos da diagonal:

$$\lambda_1 = 2, \quad \lambda_2 = 0.5$$

- Como B é diagonal,

$$x_k = B^k x_0 = \begin{bmatrix} 2^k & 0 \\ 0 & 0.5^k \end{bmatrix} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 2^k \\ 0.5^k \end{bmatrix}$$

- Componente 1: $2^k \rightarrow \infty$ quando $k \rightarrow \infty$.
- Componente 2: $0.5^k \rightarrow 0$ quando $k \rightarrow \infty$.

Ou seja, o sistema **explode** na primeira direção (autovalor > 1) e **decai** na segunda (autovalor < 1).

Diagonalização de Matrizes – Exercícios 8 a 10

Exercício 8 – Diagonalização explícita

Use novamente

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$$

com autovalores $\lambda_1 = 3, \lambda_2 = 5$ e autovetores correspondentes

$$v_1 = \begin{bmatrix} 1 \\ -1 \end{bmatrix}, \quad v_2 = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

- Monte a matriz P de autovetores.
- Monte a matriz diagonal D .
- Calcule P^{-1} .
- Verifique que $A = PDP^{-1}$.

Resposta:

- Matriz P com autovetores nas colunas:

$$P = \begin{bmatrix} 1 & 1 \\ -1 & 1 \end{bmatrix}$$

b) Matriz diagonal D com os autovalores:

$$D = \begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix}$$

c) Para calcular P^{-1} , primeiro o determinante de P :

$$\det(P) = 1 \cdot 1 - 1 \cdot (-1) = 1 + 1 = 2$$

Fórmula de inversa de 2×2 :

$$P^{-1} = \frac{1}{\det(P)} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

d) Verificar PDP^{-1} :

Primeiro calcule PD :

$$PD = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 3 & 0 \\ 0 & 5 \end{bmatrix} = \begin{bmatrix} 1 \cdot 3 + 1 \cdot 0 & 1 \cdot 0 + 1 \cdot 5 \\ -1 \cdot 3 + 1 \cdot 0 & -1 \cdot 0 + 1 \cdot 5 \end{bmatrix} = \begin{bmatrix} 3 & 5 \\ -3 & 5 \end{bmatrix}$$

Agora,

$$PDP^{-1} = \begin{bmatrix} 3 & 5 \\ -3 & 5 \end{bmatrix} \cdot \frac{1}{2} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}$$

Multiplicando:

$$\begin{bmatrix} 3 & 5 \\ -3 & 5 \end{bmatrix} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 \cdot 1 + 5 \cdot 1 & 3 \cdot (-1) + 5 \cdot 1 \\ -3 \cdot 1 + 5 \cdot 1 & -3 \cdot (-1) + 5 \cdot 1 \end{bmatrix} = \begin{bmatrix} 8 & 2 \\ 2 & 8 \end{bmatrix}$$

Dividindo por 2:

$$PDP^{-1} = \frac{1}{2} \begin{bmatrix} 8 & 2 \\ 2 & 8 \end{bmatrix} = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix} = A$$

Logo, $A = PDP^{-1}$, e A é de fato diagonalizável.

Exercício 9 – Matriz não diagonalizável

Considere

$$J = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}$$

- Encontre o(s) autovalor(es) de J .
- Encontre os autovetores associados.
- Explique por que J não é diagonalizável.

Resposta:

a) Polinômio característico:

$$\det(J - \lambda I) = \det \begin{bmatrix} 1 - \lambda & 1 \\ 0 & 1 - \lambda \end{bmatrix} = (1 - \lambda)(1 - \lambda) - 0 \cdot 1 = (1 - \lambda)^2$$

Logo, o único autovalor é

$$\lambda = 1$$

com multiplicidade algébrica 2.

b) Autovetores: resolver $(J - I)v = 0$:

$$J - I = \begin{bmatrix} 1 - 1 & 1 \\ 0 & 1 - 1 \end{bmatrix} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$$

Sistema:

$$0 \cdot x + 1 \cdot y = 0 \Rightarrow y = 0$$

O vetor é da forma:

$$v = \begin{bmatrix} x \\ 0 \end{bmatrix} = x \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

Logo, o espaço de autovetores é:

$$E_1 = \left\{ \alpha \begin{bmatrix} 1 \\ 0 \end{bmatrix} : \alpha \in \mathbb{R} \right\}$$

de dimensão 1.

- c) Como a multiplicidade algébrica do autovalor 1 é 2, mas a multiplicidade geométrica (dimensão do espaço de autovetores) é 1, não existem 2 autovetores linearmente independentes.

Portanto, não é possível formar uma matriz P invertível com autovetores nas colunas, e J não é diagonalizável.

Exercício 10 – Verificação computacional com NumPy (opcional)

Use Python/NumPy para:

- a) Calcular autovalores e autovetores de

$$A = \begin{bmatrix} 4 & 1 \\ 1 & 4 \end{bmatrix}$$

- b) Construir P e D a partir da saída do NumPy.
c) Verificar numericamente que $A \approx PDP^{-1}$.

Resposta:

```
import numpy as np

A = np.array([[4, 1],
              [1, 4]])

# 1. Autovalores e autovetores
vals, vecs = np.linalg.eig(A)
print("Autovalores:", vals)
print("Autovetores (colunas de P):")
print(vecs)

# 2. Montando P e D
P = vecs
D = np.diag(vals)

# 3. Reconstrução de A
A_rec = P @ D @ np.linalg.inv(P)
print("A reconstruída:")
print(A_rec)

# Verificação
print("Diferença A - A_rec:")
```

```
print(A - A_rec)
```